

FortiFlow

Guide complet : Revue, Déploiement & HTTPS

À propos

FortiFlow est un outil d'analyse de logs trafic FortiGate / FortiAnalyzer pour les prestations de segmentation réseau. Ce guide couvre l'ensemble du cycle de vie : analyse de code, déploiement Docker (VPS et Portainer), configuration HTTPS et maintenance.

CI : VERTE (4/4) | Déploiement : Portainer | Image : ghcr.io/tetrax/fortiflow:latest

Ce guide explique pas à pas comment déployer l'application sur la VM de production. L'application sera accessible en HTTPS sur le réseau interne à l'adresse <https://fortiflow.monentreprise.lan>.

■ Machine	■ Où ?	■ Qui ?	■ Accès
SOURCE (dev)	VPS IONOS	Da Vinci (Hermes)	Automatique (CI/CD)
DESTINATION (prod)	VM Entreprise	Valentin	Console/SSH (root)

Note

■ Déploiement interne uniquement. L'application ne sera PAS exposée sur Internet. Certificat TLS : PKI entreprise. Nom de domaine : fortiflow.monentreprise.lan.

Déroulé des opérations

1. Présentation du projet
2. ÉTAPE 1 -- Connexion à la VM de production
3. ÉTAPE 2 -- Création des répertoires persistants
4. ÉTAPE 3 -- Vérification du port 13737
5. ÉTAPE 4 -- Déploiement du stack dans Portainer
6. ÉTAPE 5 -- Vérification du bon fonctionnement
7. ÉTAPE 6 -- Mise en place du HTTPS avec certificat interne
8. Mise à jour future
9. Dépannage
10. Checklist finale

1. Présentation du projet

FortiFlow est un outil d'analyse de logs trafic FortiGate / FortiAnalyzer pour les prestations de segmentation réseau. Il importe les logs trafic, conserve leur contexte FortiGate/VDOM, construit les matrices de flux, rapproche les observations de la configuration et du routage, puis prépare une CLI FortiGate destinée à être revue par un ingénieur.

Un seul conteneur Docker : fortiflow (Node.js + Express, port interne 3737).

Note

■■ L'image Docker est déjà buildée par la CI. Rien à builder. Image : ghcr.io/tetrax/fortiflow:latest.

Revue de code synthétique

Revue réalisée le 3 août 2026 par l'équipe Da Vinci (Socrate, Archimède, Ada). 49 tests passent, 0 échec.

■ Correction	■ Impact
`node:20-alpine` → `node:22-alpine`	47 CVE corrigées
Ajout `healthcheck` Docker	Monitoring de santé du conteneur
`security_opt` + `tmpfs /tmp`	Durcissement sécurité
Rate limiting token-bucket sur `/api/upload` + `/api/admin`	Protection brute-force
Graceful shutdown (`SIGTERM`/`SIGINT`)	Arrêt propre avec fermeture du pool
Module partagé `lib/constants.js`	Déduplication `ALLOW_ACTIONS` / `DENY_ACTIONS`
Suppression `ecosystem.config.js`	Fichier mort (PM2 non utilisé)
CI : build Docker + scan Trivy + push GHCR	Pipeline de sécurité complet
`.env.example` + config Nginx	Documentation des variables et reverse proxy

2. ÉTAPE 1 -- Connexion à la VM de production

2.1 Ce que tu vas faire

Te connecter en root sur la VM de l'entreprise où tourne Portainer.

2.2 Commande

```
# Depuis ton poste :
ssh root@ADRESSE_IP_DE_LA_VM

# Vérifier Portainer :
docker ps | grep portainer
# → Doit afficher une ligne

# Vérifier Docker :
docker version
# → Doit être ≥ 24.0
```

3. ÉTAPE 2 -- Création des répertoires persistants

3.1 Pourquoi ?

Les conteneurs sont éphémères. Les données doivent survivre aux mises à jour → on crée des répertoires montés.

3.2 Commandes

```
mkdir -p /srv/fortiflow/data /srv/fortiflow/certificates
chown -R 1000:1000 /srv/fortiflow
ls -la /srv/fortiflow/
# Doit afficher data/ certificates/ avec 1000:1000
```

Note

■ Si Docker utilise un UID différent : id docker pour trouver le bon.

4. ÉTAPE 3 -- Vérification du port 13737

4.1 Pourquoi ?

L'application utilise le port 13737 en externe (mappé vers 3737 dans le conteneur). On vérifie qu'il est libre.

4.2 Commande

```
ss -tlnp | grep 13737
# RIEN affiché → OK. Sinon → choisir un autre port.
```

Repository

5. ÉTAPE 4 -- Déploiement du stack dans Portainer — déploiement via registre d'images Docker

5.1 Ce que tu vas faire

Créer un stack Portainer en mode Repository. Il télécharge automatiquement l'image depuis ghcr.io et démarre le conteneur.

5.2 Ouvrir Portainer

```
https://ADRESSE_IP_DE_LA_VM:9443
# Se connecter
```

5.3 Créer le stack

1. Menu gauche → "Stacks"
2. "+ Add stack"
3. Name : fortiflow
4. Build method : Repository
5. Repository URL : <https://github.com/Tetrax/FortiFlow>
6. Repository reference : refs/heads/main
7. Compose path : docker-compose.portainer.yml

Le fichier docker-compose.portainer.yml référence l'image ghcr.io/tetrax/fortiflow:latest -- Portainer la télécharge automatiquement.

5.4 Variables d'environnement

Section Environment variables → + Add environment variable pour chaque ligne :

Nom Valeur

```
PORT 3737
DOMAIN devval.com
MAX_UPLOAD_SIZE_MB 2048
MAX_DECOMPRESSED_SIZE_MB 4096
MAX_ARCHIVE_ENTRIES 100
MAX_XLSX_SIZE_MB 100
MAX_WORKSPACE_UNCOMPRESSED_MB 1024
MAX_SESSION_DEDUPE_KEYS 2000000
MAX_ANALYSIS_WORKERS 1
MAX_ANALYSIS_QUEUE 3
ANALYSIS_WORKER_MEMORY_MB 0
SSL_KEY /certs/privkey.pem
SSL_CERT /certs/fullchain.pem
```

Note

■ Les variables SSL_KEY et SSL_CERT sont déjà dans le compose. Ne pas les dupliquer dans Portainer sauf si tu les surcharges.

5.5 Déployer

Cliquer Deploy the stack. Portainer : 1) Pull ghcr.io 2) Crée fortiflow 3) Démarre (~15s).

6. ÉTAPE 5 -- Vérification

6.1 Depuis Portainer

Containers → le conteneur doit être running (healthy) :

```
fortiflow running (healthy)
```

6.2 Depuis le terminal SSH root

```
docker ps --filter "name=fortiflow"
curl -s http://localhost:13737/ | head -10
# → Doit afficher du HTML avec "FortiFlow"
docker logs fortiflow --tail 30
```

6.3 Depuis un navigateur (réseau interne)

```
http://ADRESSE_IP_DE_LA_VM:13737
# → Dashboard FortiFlow
```

7. ÉTAPE 6 -- HTTPS avec certificat interne

7.1 Contexte

L'URL finale sera : <https://fortiflow.monentreprise.lan>

Certificat fourni par la PKI interne de l'entreprise (AD CS, EJBCA, etc.). Pas de Let's Encrypt.

7.2 Obtenir le certificat

Demander à l'équipe IT les deux fichiers pour fortiflow.monentreprise.lan (ou wildcard *.monentreprise.lan) :

1. Certificat public : fortiflow.lan.crt

2. Clé privée : fortiflow.lan.key

Transférer vers la VM :

```
scp fortiflow.lan.crt root@VM_IP:/srv/fortiflow/certificates/  
scp fortiflow.lan.key root@VM_IP:/srv/fortiflow/certificates/
```

7.3 Placer les certificats

```
cd /srv/fortiflow/certificates  
ls -la # Doit afficher fortiflow.lan.crt et fortiflow.lan.key  
chmod 600 fortiflow.lan.key  
chmod 644 fortiflow.lan.crt  
chown -R 1000:1000 /srv/fortiflow/certificates
```

7.4 Configurer Nginx

```
nano /etc/nginx/sites-available/fortiflow
```

Coller ce contenu :

```
server {  
    listen 443 ssl http2;  
    server_name fortiflow.monentreprise.lan;  
  
    ssl_certificate /srv/fortiflow/certificates/fortiflow.lan.crt;  
    ssl_certificate_key /srv/fortiflow/certificates/fortiflow.lan.key;  
  
    ssl_protocols TLSv1.2 TLSv1.3;  
    ssl_ciphers HIGH:!aNULL:!MD5;  
  
    client_max_body_size 2048m;  
    proxy_read_timeout 3600s;  
  
    location / {  
        proxy_pass http://127.0.0.1:13737;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto https;  
        proxy_http_version 1.1;  
        proxy_set_header Upgrade $http_upgrade;  
        proxy_set_header Connection "upgrade";  
    }  
}
```

7.5 Activer

```
ln -s /etc/nginx/sites-available/fortiflow /etc/nginx/sites-enabled/  
nginx -t && systemctl reload nginx
```

7.6 DNS interne

Demander à l'équipe IT un enregistrement DNS interne :

```
fortiflow.monentreprise.lan → IP_VM
```

Note

■ En attendant : ajouter IP_VM fortiflow.monentreprise.lan dans /etc/hosts de ton poste.

7.7 Vérification HTTPS

```
https://fortiflow.monentreprise.lan
# → Dashboard en HTTPS, cadenas vert (PKI entreprise reconnue)
```

7.8 Alternative sans Nginx (TLS géré par l'application)

Ajouter ces variables dans le stack Portainer :

```
SSL_KEY=/certs/privkey.pem
SSL_CERT=/certs/fullchain.pem
```

Puis Stacks → Pull and redeploy.

Note

■ ■ Avec TLS direct, l'application écoute en HTTPS sur le port 13737 (plus HTTP). Le test curl doit devenir : `curl -k https://localhost:13737`.

Les certificats doivent être dans `/etc/ssl/fortiflow/` sur l'hôte -- le conteneur les monte en `/certs/` (lecture seule). Le serveur détecte automatiquement leur présence et bascule en HTTPS.

Upgrade Path

8. Mise à jour future — procédure de montée de version

8.1 Portainer (recommandé)

```
Stacks → fortiflow → Pull and redeploy → Update
```

Portainer pull `ghcr.io/tetrax/fortiflow:latest` et redémarre (~5 sec d'interruption).

8.2 SSH (plan B)

```
cd /srv/fortiflow
docker compose -f docker-compose.portainer.yml pull
docker compose -f docker-compose.portainer.yml up -d
```

Le compose référence maintenant `ghcr.io` directement → `docker compose pull` fonctionne.

9. Dépannage

■ Problème	■ Vérification / Action
Stack ne se déploie pas	Vérifier toutes les variables : PORT, DOMAIN, MAX_UPLOAD_SIZE_MB, etc.
Conteneur en erreur	<code>`chown -R 1000:1000 /srv/fortiflow` puis `docker logs fortiflow`</code>

■ Problème	■ Vérification / Action
Statut healthy absent	Vérifier que le healthcheck est présent dans le compose
HTTPS ne fonctionne pas	Vérifier les chemins dans Nginx, `nginx -t`, vérifier le DNS interne
Certificat non reconnu	Vérifier que le poste fait confiance à la CA racine de l'entreprise
Dashboard page blanche	`docker logs fortiflow --tail 50`, vérifier les variables d'environnement
Erreur `ANALYSIS_CANCELLED`	Analyse interrompue — relancer l'upload
Erreur `DECOMPRESSED_SIZE_LIMIT`	Archive trop volumineuse — augmenter `MAX_DECOMPRESSED_SIZE_MB`
Portainer : `no such image`	Utiliser la méthode Repository (pas d'import manuel)
Permission denied sur /certs	`chmod 644` sur les .pem dans `/etc/ssl/fortiflow/`

10. Checklist finale

- /srv/fortiflow/ créé avec permissions/ownership 1000:1000
- Port 13737 libre
- Stack Portainer déployé, conteneur healthy
- curl http://localhost:13737 fonctionne
- Certificats dans /srv/fortiflow/certificates/ (ou /etc/ssl/fortiflow/)
- Nginx configuré pour HTTPS sur fortiflow.monentreprise.lan
- DNS interne créé
- https://fortiflow.monentreprise.lan accessible, cadenas vert

■ Déploiement terminé. Application accessible en HTTPS sur le réseau interne. ■

■ Ressources

- Repo : <https://github.com/Tetrax/FortiFlow>
- Image : ghcr.io/tetrax/fortiflow:latest
- CI : <https://github.com/Tetrax/FortiFlow/actions>